



Authors & Contributors:

Nurjahan Ahmed Shiah,
Uzma Alam,
Muhammad Fahad Sohail,
Nusayba Hossain,
Ali Shandhor,
Saaram Ahmed Mustafa

Last updated: June 12, 2026
Status: Phase 01

Contents

1.	Purpose	-----	03
2.	Background Reading	-----	03
3.	Context		
	3.1 Problem Statement	-----	04
	3.2 Motivation	-----	04
	3.3 Business Goals	-----	05
	3.4 Prior related work	-----	05
	3.5 Scope and Constraints	-----	05
4.	Detailed Design		
	4.1 System Architecture	-----	07
	4.2 How users interact with the system	-----	08
	4.3 How software components interact	-----	10
	4.4 How data flows through the system	-----	10
5.	Implementation Plan	-----	11
	5.1 Phase 1: Foundation & Infrastructure	-----	11
	5.2 Phase 2: Authentication & API Gateway	-----	12
	5.3 Phase 3: Frontend Development	-----	12
	5.4 Phase 4: Analytics & Recommendation	-----	13
	5.5 Proposed Technologies	-----	13
	5.6 Source Code Management	-----	14
	5.7 Components Integration	-----	14
	5.8 Testings	-----	15
	5.9 Deployment	-----	16
6.	Project Planning		
	6.1 Team Organization	-----	16
	6.2 Big User Stories and Use-Cases	-----	17
	6.3 Development Milestones	-----	19
	6.4 Feature Ownership & Milestone details	-----	20
	6.5 Coordination	-----	28
	6.6 Development Schedule	-----	28
7.	Appendix	-----	30

Purpose

Students at universities depend upon multiple disparate ways to be able to collaborate in an academic manner. Group chats, shared drives, Discord servers, and calendars are all connected in an ad hoc manner so that study groups can be arranged, resources can be shared, and sessions can be scheduled. However, while these different tools have been useful, they have led to missed communications, lost files, and poor visibility into who is studying what and when. At this time, there is no singular platform available that is course-aware, role-structured, and smart enough to automatically connect students with the right collaborators successfully.

StudySync provides the solution to this problem with the creation of a purpose-built Collaboration Database for academic workflow in universities. With StudySync, students will be able to discover and join course-associated study groups, share resources, attend scheduled sessions, and receive recommendations for groups based upon machine learning (ML), which continuously learns from the actions of all users within StudySync. The group leaders will be able to manage their study groups, sessions, and announcements all from a single platform, while the administrators will be able to maintain the integrity of StudySync through a separate administration console. The entire academic collaboration workflow is managed by one single system based on a PostgreSQL transactional database and a Delta Lake analytical layer, allowing for all academic collaboration workflows to be consolidated into one place and removing the need for any other unrelated tool.

Background Reading

- [1] Kalideen, M. R., and C. Yağlı. "Machine Learning-based Recommendation Systems: Issues, Challenges, and Solutions." *Journal of Information and Communication Technology*, vol. 2, special issue, Jan. 2025, pp. 6-12. South Eastern University of Sri Lanka, www.seu.ac.lk/jict/publication/vl02sp01/6-12%20ML_Recommendations.pdf.
- [2] Ansmink, Sam. "Native Delta Lake Support in DuckDB." *DuckDB*, 10 June 2024, duckdb.org/2024/06/10/delta.
- [3] Victor, Uma. "FastAPI RBAC - Full Implementation Tutorial." *Permit.io*, 5 Feb. 2025, www.permit.io/blog/fastapi-rbac-full-implementation-tutorial.
- [4] Letaw, Lara. "Unified Modeling Language Class and Sequence Diagrams." *Handbook of Software Engineering Methods*, Oregon State University, 2024, <https://open.oregonstate.edu/setextbook/chapter/uml-class/>.

- [5] Armbrust, Michael, et al. "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics." *CIDR 2021*, 2021, cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf.
- [6] Jones, M., et al. "JSON Web Token (JWT)." *RFC 7519*, Internet Engineering Task Force, May 2015, datatracker.ietf.org/doc/html/rfc7519.
- [7] Percival, Harry J. W., and Bob Gregory. *Architecture Patterns with Python*. O'Reilly Media, 2020, www.cosmicpython.com/book/preface.html.
- [8] *PostgreSQL 16 Documentation*. The PostgreSQL Global Development Group, 2023, www.postgresql.org/docs/16/.

Context

Problem Statement

University students frequently struggle with three interconnected challenges when forming and managing study groups:

1. **Discovery Problem:** Students don't know which classmates share their courses or study interests, making it difficult to find compatible study partners.
2. **Coordination Problem:** Without a dedicated platform, groups rely on fragmented communication channels (WhatsApp, Discord, email) that lack structure, making it hard to schedule sessions, track attendance, or organize study materials.
3. **Resource Fragmentation:** Study materials are scattered across multiple platforms and chat histories, making it difficult for students to locate notes, textbooks, or past session recordings when they need them.

Existing tools like Discord, GroupMe, and Google Classroom were not designed specifically for academic collaboration. They are either too generic (Discord, GroupMe) or too instructor-centric (Google Classroom), missing the student-to-student peer learning use case.

Motivation

This problem emerged from the team's own experience as York University students. We noticed that despite being in the same courses, we didn't naturally connect with peers until late in the semester, if at all. Many valuable study sessions never happened because the coordination friction was too high.

StudySync was created to reduce this friction by:

- **Making discovery frictionless:** Use course enrollment to automatically surface relevant peer groups
- **Centralizing coordination:** Schedule sessions, upload resources, and discuss in one place designed for student collaboration
- **Personalizing recommendations:** Use student activity patterns to suggest groups that maximize learning value

Business Goals

StudySync is designed as a proof-of-concept for a university-wide student collaboration platform. Success is measured by:

1. **Functional Completeness:** All user stories implemented and demonstrated working end-to-end.
2. **System Quality:** Robust backend APIs, responsive frontend, and reliable data handling under realistic usage
3. **Architectural Clarity:** Demonstrate a distributed, service-oriented design that scales beyond a single course
4. **Team Execution:** Six developers working on independent features in parallel, with clean integration boundaries

Prior Related Work

1. Project Proposal (May 2026) The original project proposal defined the problem scope, selected core technologies (FastAPI, Next.js, PostgreSQL), and outlined a feature roadmap. The proposal identified the core value proposition:

- Student-led groups (not instructor-managed)
- Course-scoped recommendations (not a generic social platform)
- File sharing and session scheduling (not just discussion)

2. Architectural proposal (June 2026): We have decided to move into microservices style rather than monolithic. This influenced the design to use separate Docker containers per feature service (one each for auth, users, groups, sessions, resources, courses, admin, and recommendations) to better demonstrate service-oriented principles and prepare for future multi-team development.

Scope and Constraints

In Scope (10 user stories across 6 features):

- User authentication with role-based access control

- Study group discovery and management (creation, joining, membership)
- Session scheduling and calendar view
- File upload, storage, and download
- In-app notifications and announcements
- Personalized group recommendations using course overlap
- Admin moderation capabilities

Out of Scope (future enhancements):

- Video conferencing integration (would require Zoom/Jitsi API)
- Real-time chat (beyond async notifications and announcements)
- Mobile native apps (web-only in MVP)
- Advanced ML features (e.g., skill-based matching, learning outcome prediction)
- Integration with university authentication systems (using simple local JWT for now)
- Paid features or subscription management

Constraints:

- **Infrastructure:** Docker Compose for development and deployment; cloud hosting if course credits are available.
- **Data:** Only synthetic test data; no real student records

Detailed Design

The overall system architecture

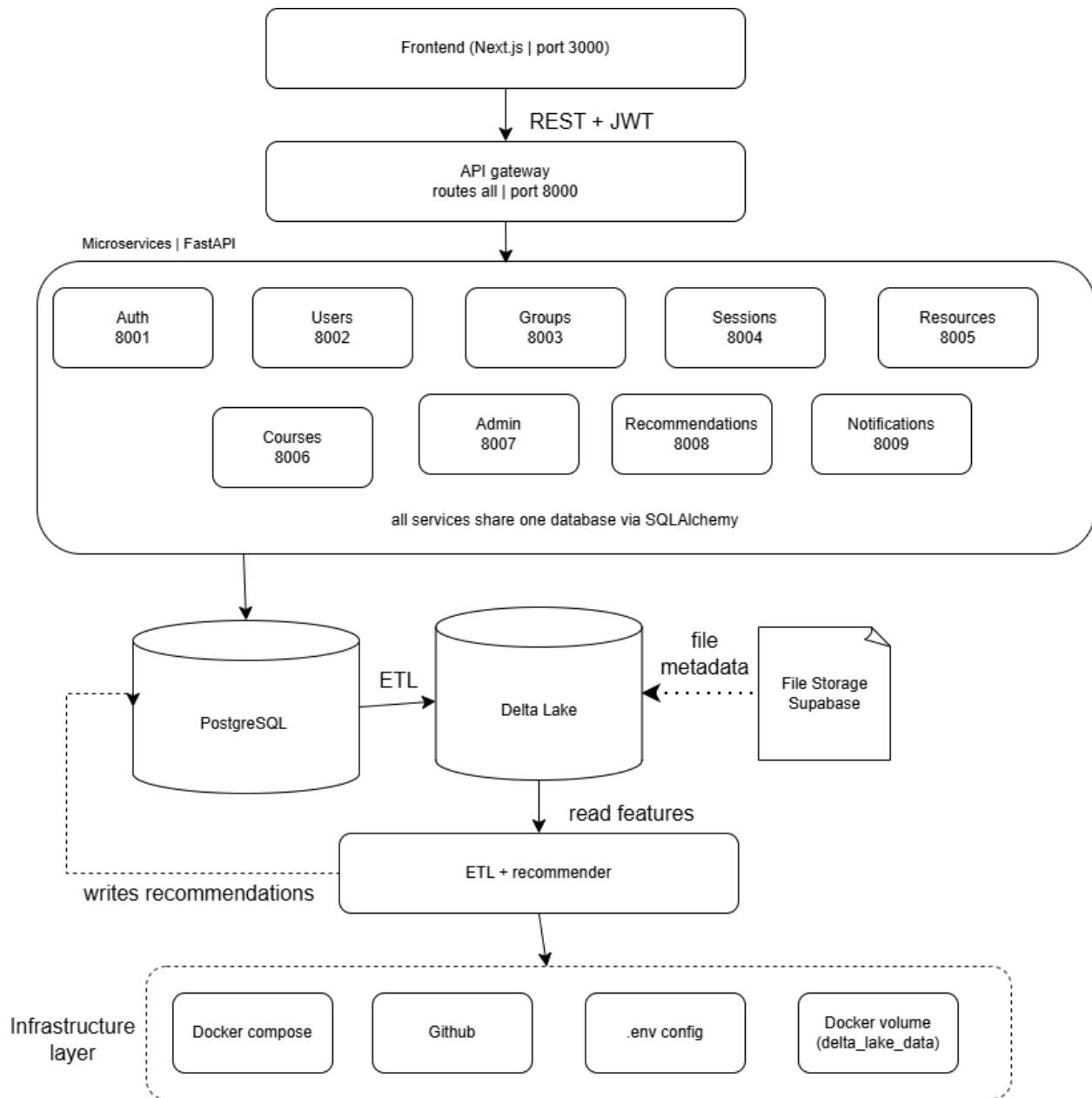


Figure 1: System Architecture

How users interact with the system

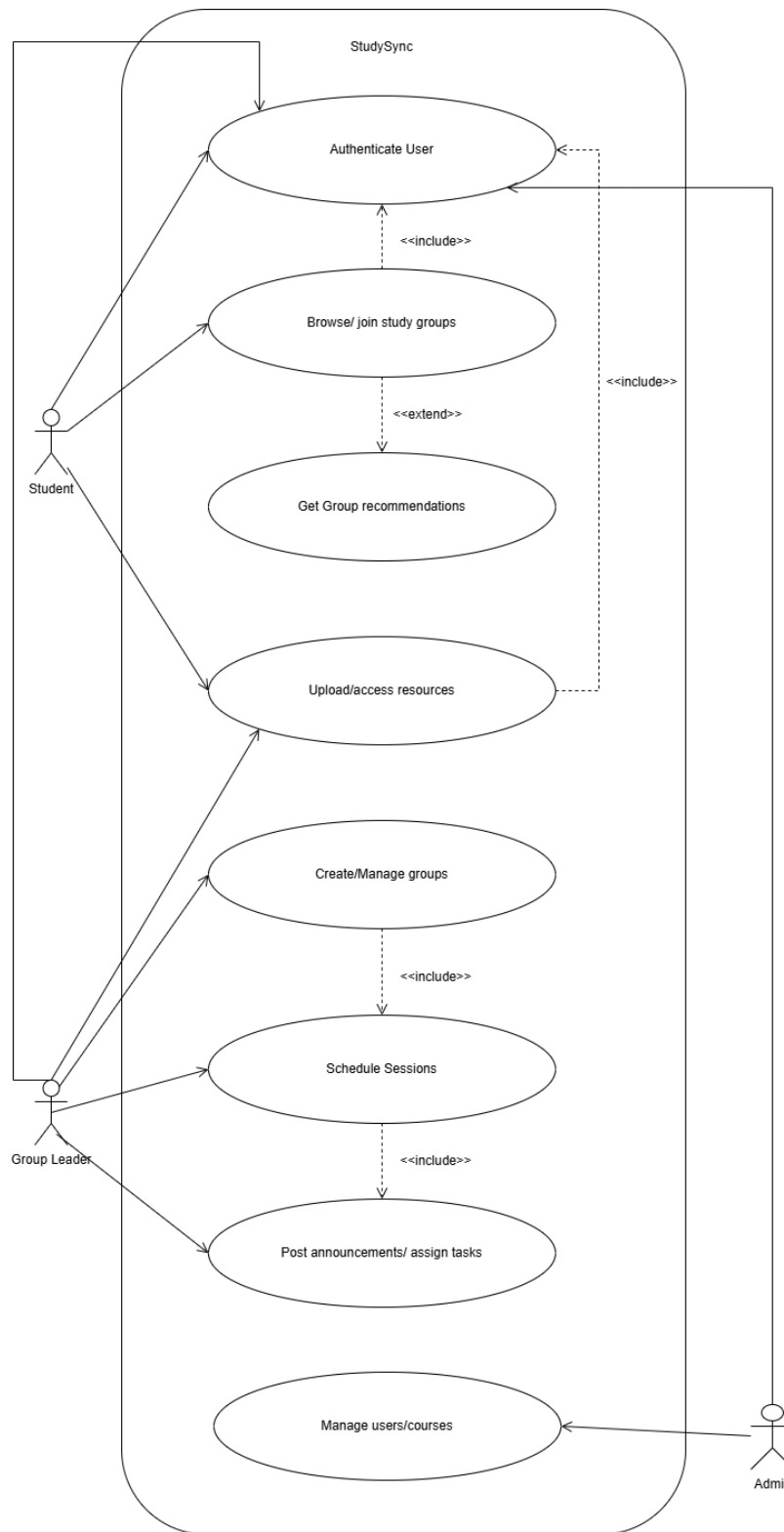


Figure 2: Use-Case Diagram

Users interact with StudySync through a responsive web interface built with Next.js 16 and React 19, accessible from any modern browser on desktop or mobile. All interaction begins at the authentication layer where new users register with their university credentials and existing users log in via a JWT-secured login form. Upon successful authentication, the system identifies the user's role, such as Student, Group Leader, or Admin, and routes them to their respective dashboard.

Students: A student's personal home page contains information and links to find study groups based on the classes they are taking as well as their activity on the platform. From this page, students can look for study groups by browsing or searching using various criteria such as class codes, subject, or calendar. Joining a study group is done with one action, and that is once they are part of a study group, students can access the study group's library of shared materials, sessions scheduled for the future, and the discussion board of the group. Students may add their notes or other documents in any format using the file upload interface. Files uploaded by the student into the library will be saved in Supabase and linked back to that study group. Students will receive a notification when there is a scheduled session, and they can view all the information about upcoming sessions, including the time and place the session will occur, as well as who will be attending.

Group Leaders: Expanded control sets are obtained by Group Leaders as well as Group Members via a group leader management panel, which provides enhanced overall control of the student experience. After creating a study group and assigning the group to one or more courses, group leaders will use the group leader management panel (a web-based application) to perform such functions as, scheduling sessions using a calendar-like interface, posting announcements to all of the group members, assigning tasks to specific group members, moderating resources that have been uploaded to the study group, and checking the live roster of the group member and managing the group membership from the group leader management panel.

Admins: Admins access a completely separate administrative console that is not included in either the student or the group leader or members' experience on the platform. From the Administrator console, it is possible for administrators to view and manage all registered users, courses, and groups throughout the platform in addition to having the ability to suspend or remove a user, create or archive a course, and review previously flagged material. The administrator interface is secured by an additional role-checking mechanism, which is an additional check at the routing level, so as to ensure that it is totally inaccessible to any non-administrator account, regardless of URL manipulation.

How software components interact

StudySync uses a service-oriented structure where the Next.js/React frontend communicates with a FastAPI backend through REST API calls. The backend routes requests to feature-focused services such as User, Courses, Study Group, Session, Resource, Notification, and Recommendation. These services handle their own domain logic while using PostgreSQL for transactional data, Supabase for uploaded files, and the recommendation pipeline for personalized study group suggestions.

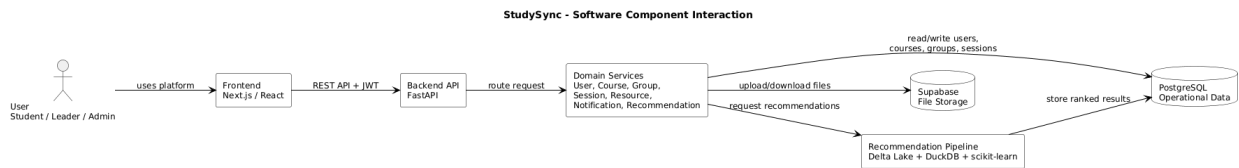


Figure 3: How software components interact

How data flows through the system

Data flow begins when a user performs an action in the frontend, such as registering, joining a group, uploading a resource, scheduling a session, or viewing recommendations. The FastAPI backend validates the request, checks JWT authentication and role permissions, and forwards it to the correct service. Operational records are stored in PostgreSQL, uploaded files are stored in Supabase, and recommendation-related activity is extracted into Delta Lake for analytical processing with DuckDB and scikit-learn before ranked recommendations are written back to PostgreSQL.

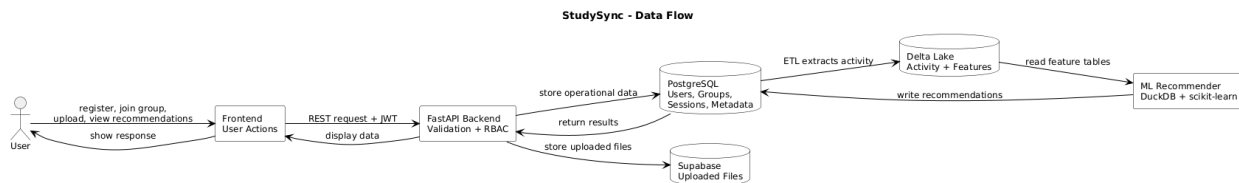


Figure 4: How data flows through the system

Implementation Plan

StudySync will be implemented using an incremental, milestone-based development approach centred around a microservices architecture. The system will be divided into independent services responsible for authentication, users, groups, courses, notifications, sessions, resources, recommendations, and administration. Each service will focus on a single domain responsibility and communicate through RESTful APIs.

The implementation will follow a dependency-based sequence to reduce integration complexity and allow the team to validate system functionality incrementally. Foundational components such as the database schema, authentication, and backend APIs will be developed first since later components depend on them. Once backend services are stable, frontend integration will begin, followed by recommendation processing, system testing, and deployment activities.

The system will be containerized using Docker and coordinated through Docker Compose to ensure a consistent development and deployment environment across all team members. Development will occur iteratively, with milestone-based integration and testing to validate functionality before moving to later phases.

Implementation work will be distributed across team members based on technical responsibilities to support parallel development and efficient integration. Development will follow a milestone-based schedule beginning with infrastructure and backend services, followed by frontend integration, recommendation processing, testing, and deployment. Detailed team organization, work breakdown, and milestone planning are provided in the Project Planning section.

The implementation process will occur in the following stages:

Phase 1: Foundation and Infrastructure

The first phase focuses on setting up the development environment and core infrastructure required for the system. Tasks include:

- GitHub repository setup
- Docker Compose configuration
- PostgreSQL database setup
- Initial project structure for frontend and backend services
- Environment variable configuration
- Database schema creation using SQLAlchemy and Alembic migrations

This phase ensures that all team members work in a consistent environment and that the project foundation is stable before feature development begins.

Phase 2: Authentication and Core Backend Services

Authentication functionality will be implemented before other protected services because user identity and access control are required throughout the platform. This phase includes:

- User registration and login
- Password hashing using bcrypt
- JWT token generation and validation
- Role-based access control (Student, Group Leader, Admin)

Once authentication is functional, the remaining core backend services will be developed in dependency order:

- Admin Service
- User Service: profile management and course enrollment
- Course Service: course catalogue and administration
- Group Service: creating and joining study groups
- Session Service: scheduling study sessions
- Resource Service: uploading and managing study materials
- Recommendation Service: personalized study group recommendations
- Notification Service

Phase 3: Frontend Development

Once backend APIs are operational, frontend pages will be integrated with backend services.

Frontend implementation order:

1. Login and signup pages
2. User dashboard
3. Group browsing page
4. Group detail and management pages
5. Study session scheduling interface
6. Resource upload and viewing functionality
7. Recommendation dashboard panel
8. Admin management pages

The frontend will communicate with backend services using REST APIs and JWT-based authentication.

Phase 4: Analytics and Recommendation Integration

Once sufficient user activity data is available, the recommendation system will be integrated into the platform. The recommendation process includes:

1. Extracting user activity data from PostgreSQL
2. Transforming data through an ETL pipeline into Delta Lake
3. Processing recommendation features using DuckDB and Python
4. Generating ranked study group recommendations
5. Writing recommendations back to PostgreSQL
6. Displaying recommendations through the frontend dashboard

This phased implementation approach ensures dependencies are completed before advanced functionality is introduced and allows testing to occur continuously throughout development.

Proposed Technologies

StudySync will use a modern full-stack technology stack designed to support scalability, maintainability, and collaborative development.

Layer	Technology	Purpose
Frontend	Next.js 16 + React + TypeScript	User interface and routing
Styling	Tailwind CSS	Responsive and reusable UI styling
Backend	FastAPI (Python)	REST API and business logic
Authentication	JWT, python-jose, passlib bcrypt	Secure authentication and authorization
Database	PostgreSQL 16	Transactional data storage
ORM	SQLAlchemy	Database interaction and modeling
Database Migration	Alembic	Version-controlled schema changes
Data Lake	Delta Lake + Apache Parquet	Analytical storage for recommendation features
Analytics	DuckDB, pandas, scikit-learn	ETL and recommendation processing

File Storage	Supabase	Storage of PDFs and uploaded study materials
DevOps	Docker + Docker Compose	Containerized development and deployment
Version Control	Git + GitHub	Team collaboration and source control

These technologies were selected because they support a distributed architecture, simplify development, and integrate well with Python-based analytics and recommendation systems.

Source-Code Management

Source code will be managed using Git and GitHub to support version control, team collaboration, and controlled integration of new features.

The repository will follow the following branching strategy:

- **main**: Contains stable, production-ready code. Direct pushes are restricted to maintain stability.
- **feature/<name>**: Used for individual feature development (e.g., feature/authentication, feature/group-management).
- **fix/<name>**: Used for bug fixes and maintenance tasks.

Development Workflow:

1. A team member creates a **feature branch** from main.
2. The feature is implemented with frequent commits following a structured commit naming convention.
3. A **Pull Request (PR)** is opened into main with a clear description of the implemented feature or fix.
4. Other team members review the changes, provide feedback if necessary, and approve the pull request.
5. Once approved, the changes are merged into main.

This workflow helps maintain code quality, reduces merge conflicts, and ensures all major changes are reviewed before being added to the main codebase.

Components Integration

Every single microservice is built independently inside its respective Docker container with a dedicated API route for integration into the application through well-defined API contracts instead of any code that needs to be shared. For Phase 0, team members will define their

endpoints, request format, and the response format of each service to ensure a reliable and stable API interface for parallel work.

The integration happens gradually throughout the whole process. Once services reach milestone M3 (backend complete), they get connected to the API gateway, and relevant frontend features become part of the project by reaching milestone M4. The Next.js frontend communicates with all backend services through the API gateway using authenticated REST API requests, making the gateway the central integration point of the system.

Some shared components like PostgreSQL, Supabase File Storage, and Delta Lake volumes can be used in many services at once. In order to maintain consistency in database operations, we use Alembic migrations where every individual developer is accountable for their tables and migration scripts.

System-wide integration testing takes place on Phase 5 through deploying all services using Docker Compose and making sure that the processes work properly across service boundaries. This testing ensures that services communicate properly, data is exchanged consistently, and the system functions as an integrated whole.

Testings

Testing is organized around the milestone schedule, so each milestone has its own clear set of tests.

Unit Testing (M3 - Completion of All Backend APIs): By the end of milestone 3, all members are expected to write basic unit tests for their individual services using pytest. Unit tests check whether a module functions properly when operated under optimal conditions (such as proper login and valid data). They also determine how the module handles errors (such as faulty inputs, invalid permissions, and lack of data).

Recommendation Pipeline Testing (M5 - Recommendations Live): After incorporating the recommendation system, it should be confirmed that the ETL pipeline works with real seeded data and generates a list of recommendations of the group. The recommendations provided by the recommendations endpoint should be consistent with the recommendations appearing in the dashboard.

Integration Testing (M6 - Testing Complete): This is where we check that all the services actually talk to each other properly through the API gateway and that data stays consistent in PostgreSQL. Major integration tests include the whole process flow of the user, which involves registration, participation in a group, upload of resources, session scheduling, and notification receipt. The other important test includes testing the whole process flow of the administrator,

where an administrator logs in, deletes resources, checks that these resources were deleted from the database, and that they are logged into the moderation history.

End-to-End Smoke Testing (M6): This testing checks the fundamental capabilities of the system through the simulation of a real user process including user account creation, group membership, uploading the resources, and receiving a recommendation. Completion of this process proves the proper functioning of all key modules and services inside the Docker Compose ecosystem.

Continuous Integration: During the process of making pull requests into the main branch, all the unit and integration tests will run using Docker Compose automatically. In Phase 5, only bug-fix branches can be merged. At this stage, integration issues between services are identified and resolved, ensuring that all M6 acceptance criteria are satisfied before the system is considered fully integrated.

System Deployment

StudySync will be deployed using Docker and Docker Compose. The full system is brought up with a single command:

```
docker compose -f docker-compose-microservices.yml up --build
```

which starts all containers including the frontend, API gateway, eight backend microservices, and the PostgreSQL database. The access instructions and the public GitHub repository link will be provided for evaluation.

If cloud credits are made available through the course, the system will be hosted online on a cloud server (such as AWS or Google Cloud) by pulling the repository onto the server and running the same Docker Compose command, making the application accessible via a public URL.

Project Planning

Team organization

Work is divided by feature rather than by layer. Each of the six members owns one feature end-to-end (API-gateway, services, database schema, and frontend pages) so that everyone writes full-stack code and no one is siloed into only frontend or only backend.

Big User Stories & Use Cases

Each user story maps to an implementable sub-feature and assigned to a lead contributor. The lead contributor is responsible for all related work required to satisfy the acceptance criteria. Stories are the primary definition of done for each milestone.

ID	Feature	Role	User Story	Acceptance Criteria
US-01	Auth & RBAC	Student / All	As a user, I want to register and log in with a role (Student, Group Leader, Admin) so that I can access features appropriate to my role.	Registration stores hashed password and role; login returns a JWT; role-restricted endpoints reject wrong roles with 403; protected pages redirect unauthenticated users.
US-02	Study Groups	Student	As a student, I want to browse study groups filtered by my enrolled courses and join one with a single action.	Groups list filtered by course; join adds student to member list and returns updated member count; a student cannot join the same group twice.
US-03	Resources	Student	As a student, I want to upload notes or PDFs to my group and download files uploaded by other members.	File stored in Supabase; metadata (name, uploader, size, timestamp) saved in PostgreSQL; file appears in group resource list within 3 seconds; download link resolves to correct file.
US-04	Recommendations	Student	As a student, I want to see personalized study group recommendations based on my course overlap with other students.	Recommendation panel shows ≥ 3 ranked groups; scores reflect course-overlap similarity; panel updates after ETL pipeline runs; clicking a suggestion navigates to the group detail page.
US-05	Notifications	Student	As a student, I want to receive in-app notifications when a new session is scheduled or an announcement is posted in my group.	Notification created in DB on session/announcement event; unread count badge updates in UI without page refresh; marking as read persists across sessions.

ID	Feature	Role	User Story	Acceptance Criteria
US-06	Study Groups	Group Leader	As a group leader, I want to create a study group, edit its details, and remove members so that I can manage my group.	Only the creator has a leader role for that group; leader can update group name/description/course; leader can remove any member except themselves; non-leaders cannot access management controls.
US-07	Sessions	Group Leader	As a group leader, I want to schedule a study session with a date, time, and location so that members know when and where to meet.	Session saved with all fields; all group members receive a notification on creation; session appears in a list/calendar view; past sessions are visually distinguished from upcoming ones.
US-08	Notifications	Group Leader	As a group leader, I want to post announcements and assign tasks to specific members so that I can coordinate group work.	Announcements visible to all group members immediately; tasks linked to a specific member appear in their task list; both trigger a notification for affected members.
US-09	Admin Panel	Admin	As an admin, I want to view, edit, and deactivate any user account or course entry from a management dashboard.	Admin dashboard lists all users and courses with search/filter; edits persist immediately; deactivated users cannot log in; admin actions are logged with timestamp.
US-10	Admin Panel	Admin	As an admin, I want to delete any group, resource, or announcement and have that action logged.	Admin can delete any entity regardless of ownership; deleted items are no longer accessible; a moderation log entry is created for each deletion with admin ID, target, and timestamp.

Development Milestones

Milestones are verifiable checkpoints. Each has a target date and concrete success criteria that any team member can independently confirm. A milestone is only marked complete when all its criteria are met, not when work is "in progress." Slippage is flagged at the next weekly sync and Phase 6 absorbs minor delays.

Milestone	Target	Deliverable / Success Criteria
M0 - Planning Complete	Jun 13	Design document drafted; every member has defined their feature's DB schema, API endpoints, and UI components; GitHub project board active with issues per feature.
M1 - Infrastructure Ready	Jun 16	Docker Compose starts all service containers; PostgreSQL reachable; Alembic migrations run; Supabase storage bucket configured; each feature service scaffold passes a health-check endpoint.
M2 - Auth Working	Jun 20	Register, login, logout functional; JWT issued and validated; role middleware rejects wrong roles with 403; protected Next.js routes redirect unauthenticated users to login.
M3 - All Backend APIs Complete	Jun 25	Every feature's endpoints return correct responses with real DB data: groups, resources, sessions, notifications, announcements, admin controls, and recommendation stubs. Swagger docs generated.
M4 - All Frontend Pages Integrated	Jul 1	Every feature's frontend pages render real data from the backend: login, groups, resources, sessions, notifications, recommendations (stub data), admin panel. File upload and download works end-to-end.
M5 - Recommendations Live	Jul 7	ETL pipeline writes activity events to Delta Lake; scikit-learn model produces ranked suggestions; recommendation panel displays live personalized results; all others have seeded test data.
M6 - Testing Complete	Jul 14	Unit tests pass for all 6 feature services; integration tests pass across service boundaries; end-to-end smoke test passes (register → join group → upload resource → view recommendation); known bugs resolved.
M7 - Final Submission	Jul 21	One-command Docker Compose deployment succeeds from a clean checkout; documentation finalized; all milestone deliverables present.

Sub-Feature Ownership & Milestone

The table below shows who owns which feature, which user stories it covers, and the full-stack scope of the work.

Lead Contributor	Stories	Title	Scope	Milestone Checklist
	US- 01	Big User Story	Authentication, Identity & Gateway	
Shiah	US-A.1	User Registration/ Signup & Account Creation	Full signup flow: form validation, duplicate-email checks, password strength rules, account creation with role assignment (Student/Group Leader/Admin), and welcome state on first login.	M2: POST /auth/register creates user with hashed password and role stored; signup page submits form and redirects on success.
Nusayba	US-A.2	User Login, Sessions and Token Lifecycle	Complete login flow including JWT issuance, token storage, automatic token refresh before expiry, and session expiry handling (auto-logout + redirect).	M2: POST /auth/login returns signed JWT on valid credentials, 401 on invalid; login page submits form and redirects on success.
Nusayba	US-A.3	Role-Based Access Control (RBAC) System	A reusable permission system enforced on both backend and frontend - every protected route checks role (Student/Group Leader/Admin) and rejects or hides UI accordingly.	M2: JWT middleware rejects missing/expired tokens; role middleware returns 403 on mismatch; protected Next.js routes redirect unauthenticated users to /login; nav shows role-appropriate links.
Nusayba	US-A.4	API Gateway and Service Routing	The gateway is the single entry point for the frontend - design and implement routing rules that forward requests to the correct	M1: Docker Compose starts all service containers; gateway routes reachable. M3: all auth endpoints functional with role

Lead Contributor	Stories	Title	Scope	Milestone Checklist
			microservice, with consistent error formatting across all 9 services.	enforcement through the gateway.
Nusayba	US-A.5	System Health Dashboard & Logging	A diagnostic page (admin-accessible) showing live health status of every microservice, plus centralized structured logging across the backend for debugging during demo and development.	M1: each feature service scaffold passes a health-check endpoint.
	US-02, US-06	Big User Story	Study Groups (Discovery, Membership & Management)	
Ali	US-B.1	Group Discovery and Browse Page	A searchable, filterable directory of study groups - filter by course, search by name, sort by member count/activity, with pagination for large lists.	M3: GET /groups filtered by course returns correct data with Swagger docs generated. M4: /groups browse page with course filter renders real data.
Ali	US-B.2	One-Click join & Membership Management	Students join groups with a single click; the system prevents duplicate joins, updates member counts live, and lets users leave groups they've joined.	M3: POST /groups/:id/join adds student to member list and returns updated member count; duplicate-join rejected.
Ali	US-B.3	Group Creation & Configuration Wizard	A multi-step creation flow where group leaders set name, description, linked course(s), visibility (public/private), and initial settings - not just a single form.	M4: create group modal functional end-to-end.

Lead Contributor	Stories	Title	Scope	Milestone Checklist
Nusayba	US-B.4	Group Leader Management Console	A dedicated panel for group leaders to manage their group end-to-end: edit details, view full member roster with roles, remove members, and transfer/assign co-leader status.	M3: DELETE /groups/:id/members/:uid - leader can remove any member except themselves; non-leaders rejected. M4: member removal (leader only) functional.
Fahad	US-B.5	Group Detail Hub	The central page for any group - combines overview, member list, linked courses, and quick links to that group's sessions and resources into one cohesive view.	M4: /groups/:id detail page renders real data from backend.
	US- 07	Big User Story	Sessions, Scheduling & Calendar	
Uzma	US- C.1	Session Scheduling System	Group leaders create study sessions with date, time, duration, location (physical or virtual link), and description - with validation against scheduling conflicts.	M3: POST /sessions saves session with all fields; all group members receive a notification on creation.
Fahad	US-C.2	Calendar View (Month/Week)	A visual calendar showing all of a group's sessions, switchable between month and week views, with color-coding and click-to-view-details.	M4: calendar view in group detail renders real session data.
Fahad	US-C.3	Session List and Timeline View	An alternative list-based view of sessions, grouped by upcoming/past, with clear visual separation and the ability to filter by date range.	M4: session list distinguishes past vs. upcoming sessions visually.

Lead Contributor	Stories	Title	Scope	Milestone Checklist
Uzma	US-C.4	Session Editing and Cancellation Flow	Leaders can edit session details after creation or cancel a session entirely - both actions trigger notifications to all group members and update the calendar/list immediately.	M3: GET /groups/:id/sessions reflects edits/cancellations correctly.
Fahad	US-C.5	Session Detail and RSVP Page	A dedicated page per session showing full details, attendee list, and a lightweight RSVP/attendance indicator so members can mark themselves as attending.	M4: session detail page functional with attendee list.
	US-03	Big User Story	Resources, File Management & Storage	
Shiah	US-D.1	Supabase File and upload pipeline	The full upload pipeline - frontend uploads directly to Supabase storage, receives a public URL, and registers that file's metadata with the backend, with progress feedback throughout.	M3: POST /resources (multipart upload to Supabase) saves metadata in PostgreSQL. M4: file upload component in group detail works end-to-end.
Ali	US-D.2	Resource Library & Organization	A browsable library view per group showing all shared files, organized by type (PDFs, docs, images, links) with search and filtering, designed to scale to dozens of files without feeling cluttered.	M3: GET /groups/:id/resources returns metadata; file appears in group resource list within 3 seconds. M4: resource list with download links renders real data.
Fahad	US-D.3	Download and File Preview	Files can be downloaded directly, and where possible (PDFs, images), previewed inline without leaving the page - reducing friction for students browsing shared notes.	M3: download link resolves to correct file.

Lead Contributor	Stories	Title	Scope	Milestone Checklist
Nusayba	US-D.4	Resource Permissions and Deletion	Uploaders can delete their own files; group leaders and admins can delete any file in their group - all deletions are confirmed and removed from both Supabase and the database.	M3: DELETE /resources/:id enforces uploader/leader/admin permissions. M4: delete resource (uploader only) functional in UI.
Fahad	US-D.5	Upload Activity & Storage Stats	A small dashboard widget (visible to group leaders) showing recent upload activity, total files, and storage usage per group - gives leaders visibility into how their group's library is growing.	M4: stats widget renders real aggregated data.
	US05-US08	Big User Story	Notifications, Announcements & Coordination	
Ahmed	US-E.1	Notification Centre	A full notification system - bell icon with live unread count, dropdown feed showing recent notifications, and a dedicated full-page feed with read/unread states and filtering by type.	M3: GET /notifications/:userId, PATCH /notifications/:id/read. M4: notification bell with unread count; /notifications feed renders real data; unread badge updates without page refresh; read state persists across sessions.
Ahmed	US-E.2	Announcement Board	Group leaders post announcements visible to the whole group, with support for pinning important announcements and a chronological feed members can scroll through.	M3: POST /announcements triggers notification to all members; announcements visible immediately. M4: announcement form functional.
Ahmed	US-E.3	Task Assigning and Tracking	Leaders assign tasks to specific members with due	M3: POST /tasks - tasks linked to a specific member

Lead Contributor	Stories	Title	Scope	Milestone Checklist
			dates; each member sees their assigned tasks in a personal task list and can mark them complete.	appear in their task list and trigger notification. M4: task list per member renders real data.
Ali	US-E.4	Activity Feed (Group Timeline)	A chronological feed within each group showing recent activity - new members joining, sessions scheduled, resources uploaded, announcements posted - giving members a quick "what happened while I was away" view.	M4: activity feed renders merged timeline from backend.
Ahmed	US-E.5	Notification Preferences	Users can control which types of notifications they receive (sessions, announcements, tasks) via a settings panel - giving them control over notification volume.	M4: preferences page persists settings and filters notification creation accordingly.
	US04- US09- US10	Big User Story	Admin Console, Recommendations & Analytics	
Shiah	US-F.1	Admin User and Course Management Dashboard	A full management console listing all users and courses with search, filter, inline editing, and the ability to deactivate accounts - the central control panel for platform oversight.	M3: POST /admin/users/:id/deactivate - deactivated users cannot log in; admin actions logged with timestamp. M4: /admin dashboard with user/course tables renders real data.
Ahmed	US-F.2	Moderation Console and Audit Log	Admins can delete any group, resource, or announcement regardless of ownership - every deletion is recorded in a visible audit log showing admin ID, target, action, and timestamp.	M3: DELETE /admin/:entity/:id - deleted items no longer accessible; moderation log entry created. M6: admin moderation test - admin deletes resource,

Lead Contributor	Stories	Title	Scope	Milestone Checklist
				verifies inaccessible, verifies log entry.
Shiah	US-F.3	ETL Pipeline & Data Lake Integration	Build the pipeline that extracts platform activity (joins, uploads, session attendance) from PostgreSQL and writes it into Delta Lake as structured Parquet tables, ready for analytics.	M3: ETL pipeline stub runs without error. M5: ETL job extracts join/upload/view events and writes Delta Lake Parquet tables.
Shiah	US-F.4	Recommendation Engine (Course-Overlap Scoring)	A scikit-learn-based model that scores study groups for each user based on course overlap and activity similarity with other members, writing ranked results back to PostgreSQL.	M3: GET /recommendations/:userId returns placeholder response. M5: DuckDB computes course-overlap similarity matrix; scikit-learn model ranks groups and writes scores to PostgreSQL; endpoint returns live ranked results.
Ali	US-F.5	Recommendation Dashboard & Insights Panel	The student-facing panel showed ranked group recommendations with match-score visualizations, plus a lightweight "why this group" explanation based on shared courses.	M4: /recommendations panel renders ranked group cards (stub data acceptable until M5). M5: panel displays real results with match scores.
Ahmed	US-F.6	Platform Analytics Overview (Admin)	A high-level analytics page for admins showing platform-wide stats - total users, active groups, sessions scheduled this week, most-active courses - pulled from the Delta Lake analytics layer.	M5: analytics page renders aggregated stats from Delta Lake/PostgreSQL.
		AI Add-On Features	Extension	

Lead Contributor	Stories	Title	Scope	Milestone Checklist
Uzma	US- G.1	AI Study Group Matchmaking Summary	Extends F.5: Beyond the course-overlap score, generate a short natural-language explanation of why a group is a good match - e.g. "This group shares 2 courses with you and is most active in the evenings, matching your activity pattern."	
Uzma	US- G.2	Smart Session Notes Summarizer	Extends C.5: After a session, a leader or member can paste rough notes/transcript and get an AI-generated summary with key points and action items, automatically saved as a resource for the group.	
Uzma	US- G.3	AI Resource Q&A ("Ask your library")	Extends D.2: Members can ask a natural-language question (e.g. "What did we cover about normalization?") and the system searches uploaded resource text/descriptions and returns a relevant answer with links to the source files.	
Uzma	US- G.4	Announcement Drafting Assistant	Extends E.2: When a leader writes an announcement, an "improve with AI" button rewrites a rough draft into a clear, well-formatted announcement - useful for non-native speakers or quick communication.	

Lead Contributor	Stories	Title	Scope	Milestone Checklist
Shiah	US-G.5	AI Onboarding Course Suggestions	Extends A.1: During signup, after a student enters their program/year, AI suggests likely courses they're enrolled in based on common program curricula - speeding up the enrollment step.	

Coordination

Because each member owns a vertical slice of the system, coordination focuses on keeping API contracts stable so that other features can integrate cleanly.

- **Git branching:** Each feature is developed on a feature/<name> branch. PRs in main require one peer review. No direct pushes to main.
- **API contracts first:** During Phase 0, every member writes a short spec for their endpoints (method, path, request/response shape) before writing code. This prevents integration surprises in Phase 5.
- **Weekly syncs:** Team meets weekly to review progress, flag blockers, and adjust timelines. Minutes recorded in the Appendix.
- **GitHub Issues:** One issue per user story, labelled with the feature name and phase. Closed when the story's acceptance criteria are met.
- **Shared DB migrations:** All schema changes go through Alembic. Each member writes migrations only for their own tables. Migration order is reviewed at weekly syncs to avoid conflicts.

Development Schedule

Each feature service runs in its own Docker container with its own routing prefix, rather than all services sharing a single FastAPI process. Services communicate via REST across container boundaries.

Phase	Timeline	Notes
Phase 0: Planning	Week 1: Jun 9-16	All members
Phase 1: Infrastructure	Week 1-2: Jun 16-Jun 25	Shiah leads infrastructure; others scaffold their service
Phase 2: APIs	Week 2-3: Jun 25-Jul 01	Swagger docs auto-generated per service
Phase 3: Frontend	Week 3-4: Jul 01-Jul 07	Frontend integrates with each member's own API
Phase 4: Analytics	Week 4-5: Jul 07-Jul 14	Depends on Phase 2 data being seeded
Phase 5: Integration & Testing	Week 5-6: Jul 14- Jul 21	Ali coordinates; all members fix own bugs
Phase 6: Deployment & Buffer	Week 6-7: Jul 21-Aug 2	Shiah leads deployment

Notes:

- Phase 0 and Phase 1 overlap. Planning tasks target Jun 13; infrastructure tasks run through Jun 16.
- Within Phase 2-3, each member's backend and frontend can pipeline: once your endpoints are stable, start your pages. You don't need to wait for other members' backends.
- Phase 4 (Analytics) depends on Phase 2 being complete enough to have seeded real activity data. All members should have their services accepting writes by Jun 25.
- Phase 5 is a hard integration gate. Only bug-fix branches are merged after Jul 7. New features require agreement from the full team.
- Phase 6 is a buffer. If Phase 5 slips by up to one week, Phase 6 absorbs it without missing the final submission.

Appendix

EECS 4314 Meeting Minutes



Project Name	StudySync
Date & Time of Meeting	Meeting 1: 7/06/2026 11 am -12 pm Meeting 2: 11/06/2026 9:30 pm - 10:30 pm Meeting 3: 12/06/2026 7 pm- 8 pm

Attendees			
#	Name	Student ID	Username (email)
1	Nurjahan Ahmed Shiah	218802348	nshiah49@my.yorku.ca
2	Ali Shandhor	218932178	ali2030@my.yorku.ca
3	Ahmed Mustafa	218729541	ahmed779@my.yorku.ca
4	Uzma Alam	219159771	uzmaa@my.yorku.ca
5	Nusayba Hossain	219971944	nusaybah@my.yorku.ca
6	Muhammad Fahad Sohail	218664623	fadi786@my.yorku.ca

Decision Made / Agenda / Objectives / Plan for the Coming Week	
- Split up the work amongst everyone - Everyone has their share to get done by Thursday - Will meet Friday Evening to go over everything - Will change the project structure from monolith to microservices - Finalise the design document to submit	